

Fiche 301 — Vecteurs : arithmétique vectorisée, recyclage, valeurs manquantes

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 2 « Simple manipulations; numbers and vectors », §2.1 à §2.6 et §2.8
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §3.1.1 « Constants », §3.1.4 « Operators », §3.3 « Elementary arithmetic operations » (§3.3.1 recyclage, §3.3.4 traitement des <code>NA</code>)
Difficulté	Fondamental — tout le reste de R est bâti là-dessus
Temps d'étude estimé	1 h 30
Prérequis	Fiche 300 (session, assignation, mots réservés)
Concepts clés	<code>c()</code> , <code><-</code> / <code>=</code> / <code>assign()</code> / <code>-></code> , un scalaire est un vecteur de longueur 1, vectorisation, recyclage , avertissement de non-multiple, longueur nulle, <code>:</code> et sa priorité, <code>seq()</code> , <code>rep()</code> , <code>TRUE</code> / <code>FALSE</code> / <code>NA</code> , piège <code>T</code> / <code>F</code> , coercion logique $\rightarrow$ 0/1, <code>NA</code> contre <code>NaN</code> , <code>is.na()</code> contre <code>is.nan()</code> , <code>x == NA</code> , types de <code>NA</code> , <code>NA_character_</code> , chaînes et séquences d'échappement, <code>paste()</code> et <code>sep</code> , suffixe <code>L</code> , table des opérateurs, <code>`+`</code> (<code>x</code> , <code>y</code>)
À retenir en priorité	Le recyclage et son avertissement · <code>x == NA</code> ne teste rien · <code>1:n-1</code> $\neq$ <code>1:(n-1)</code> · <code>T</code> / <code>F</code> réassignables · la coercion d'un logique en <code>0</code> / <code>1</code> .

Vue d'ensemble

LE VECTEUR	la structure la plus simple : une collection ORDONNEE de nombres un nombre isole EST un vecteur de longueur 1 -- il n'y a pas de scalaire
CREER	<code>c(10.4, 5.6, 3.1)</code> <code>1:30</code> <code>seq(-5, 5, by=.2)</code> <code>rep(x, times=5)</code>
ARITHMETIQUE	element par element, TOUJOURS <code>2*x + y + 1</code> $\rightarrow$ chaque operation recycle le plus court
RECYCLAGE	longueur du resultat = longueur du PLUS LONG

le plus court est repete, EVENTUELLEMENT FRACTIONNAIREMENT
pas un multiple -> AVERTISSEMENT (mais le calcul se fait)
longueur 0 quelque part -> resultat de longueur 0

QUATRE TYPES	numerique	logique	caractere	(+ complexe)
	TRUE/FALSE/NA		"texte"	1+2i

MANQUANT	NA = non disponible	is.na()	TRUE pour NA ET NaN
	NaN = indefini (0/0, Inf-Inf)	is.nan()	TRUE pour NaN SEUL
	x == NA -> QUE DES NA. Ce n'est PAS un test.		

PIEGES	1:n-1 vaut (1:n)-1	T et F sont des VARIABLES
--------	--------------------	---------------------------

Le problème posé. « *R operates on **named data structures**. The simplest such structure is the **numeric vector**, which is a single entity consisting of an **ordered collection of numbers**.* » Tout le reste — matrices, tableaux, facteurs, listes, data frames — se construit à **partir du vecteur**, en lui ajoutant des attributs. Comprendre le vecteur, c'est comprendre 80 % de R.

INTUITION — L'ABSENCE DE SCALAIRE.

« ***A number occurring by itself in an expression is taken as a vector of length one.*** » En R, il n'existe pas de « nombre » à part : **3** est un vecteur numérique de longueur 1. Cette uniformité est ce qui rend la vectorisation possible partout, sans exception ni cas particulier — et c'est aussi la raison pour laquelle le recyclage (concept 3) s'applique **même à une constante**.

● Concept 1 — Créer un vecteur, et les quatre façons d'assigner

DÉFINITION (§2.1).

Pour créer un vecteur nommé `x`, composé de cinq nombres — 10,4 ; 5,6 ; 3,1 ; 6,4 et 21,7 — on utilise la commande :

```
x <- c(10.4, 5.6, 3.1, 6.4, 21.7)
```

*C'est une **assignation** utilisant la fonction `c()`, qui dans ce contexte peut prendre **un nombre arbitraire d'arguments vectoriels** et dont la valeur est **un vecteur obtenu en concaténant ses arguments bout à**

bout.*

Le « **dans ce contexte** » compte. La note 1 du §2.1 prévient : « *With other than vector types of argument, such as **list mode arguments**, the action of `c()` is **rather different**.* » Voir fiche 306.

1.1 Les quatre écritures de l'assignation

NOTATION (§2.1).

L'**opérateur d'assignation** `<-` est composé des deux caractères `<` (« inférieur à ») et `-` (« moins ») placés **strictement côte à côte**, et il « **pointe** » vers l'**objet qui reçoit** la valeur de l'expression.

Écriture	Ce que dit le cours
<code>x <- c(10.4, 5.6, 3.1, 6.4, 21.7)</code>	la forme usuelle
<code>x = c(...)</code>	« <i>In most contexts the <code>=</code> operator can be used as an alternative.</i> » — « dans la plupart des contextes », pas dans tous
<code>assign("x", c(...))</code>	« <i>An equivalent way of making the same assignment</i> ». <code><-</code> peut être vu comme un raccourci syntaxique de <code>assign()</code>
<code>c(10.4, 5.6, 3.1, 6.4, 21.7) -> x</code>	« <i>Assignments can also be made in the other direction</i> »

POURQUOI <- ET PAS =.

Le cours reste prudent (« dans la plupart des contextes ») parce que `=` sert aussi à **nommer un argument** dans un appel de fonction. `f(n = 10)` passe l'argument `n` ; `f(n <- 10)` crée la variable `n` et passe sa valeur en position. Deux instructions différentes à un caractère près. Et rappel de la fiche 300 : `x<-5` sans espaces est une **assignation**, `x < -5` une **comparaison**.

Ce qui suit une assignation (§2.1). « *If an expression is used as a complete command, the value is **printed and lost**.* » Ainsi, après les assignations ci-dessus :

```
1/x      # AFFICHE les cinq inverses -- x, lui, est inchangé
y <- c(x, 0, x)  # un vecteur de 11 entrees : deux copies de x, un zero au milieu
```

c() aplattit. `c(x, 0, x)` ne crée pas une structure à trois éléments dont deux sont des vecteurs : elle produit **un vecteur plat de 11 nombres**. C'est la conséquence directe de « concaténer bout à bout ».

● Concept 2 — L'arithmétique est élément par élément

DÉFINITION (§2.2).

Les vecteurs peuvent être utilisés dans des expressions arithmétiques, **auquel cas les opérations sont effectuées élément par élément**.

INTUITION — CE QUE « VECTORISÉ » VEUT DIRE (R LANGUAGE DEFINITION §3.1.4).

« R deals with **entire vectors of data at a time**, and most of the elementary operators and basic mathematical functions like `log` are **vectorized**. This means that e.g. adding two vectors of the same length will create a vector containing the **element-wise sums, implicitly looping over the vector index**. »

La boucle existe toujours — **elle est simplement écrite pour vous, en C**. C'est pourquoi, en R, une boucle `for` explicite sur les éléments d'un vecteur est presque toujours le signe qu'on n'a pas trouvé l'opération vectorisée correspondante.

2.1 Les opérateurs et les fonctions élémentaires

Les opérateurs arithmétiques élémentaires sont les habituels `+`, `-`, `*`, `/` et `^` pour l'élévation à une puissance. En outre, **toutes les fonctions arithmétiques courantes sont disponibles** : `log`, `exp`, `sin`, `cos`, `tan`, `sqrt`, etc. (§2.2)

Fonction (§2.2)	Ce qu'elle rend
<code>max(x)</code> / <code>min(x)</code>	le plus grand / le plus petit élément
<code>range(x)</code>	« a vector of length two, namely <code>c(min(x), max(x))</code> »
<code>length(x)</code>	le nombre d'éléments
<code>sum(x)</code> / <code>prod(x)</code>	le total / le produit
<code>mean(x)</code>	la moyenne d'échantillon — « the same as <code>sum(x)/length(x)</code> »
<code>var(x)</code>	la variance d'échantillon , c'est-à-dire <code>sum((x-mean(x))^2)/(length(x)-1)</code>
<code>sort(x)</code>	un vecteur de même taille , éléments par ordre croissant
<code>order(x)</code> , <code>sort.list(x)</code>	« more flexible sorting facilities » : elles produisent une permutation qui réalise le tri
<code>pmax(...)</code> , <code>pmin(...)</code>	les maxima et minima parallèles

$$\text{var}(x) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

COMPLÉMENT — VAR() SUR UNE MATRICE (§2.2).

« If the argument to `var()` is an ***n-by-p matrix*** the value is a ***p-by-p sample covariance matrix*** got by regarding the ***rows as independent p-variate sample vectors***. » Une même fonction, deux comportements selon la forme de l'argument — le premier exemple, dans le cours, de ce que les attributs (fiche 303) changent au résultat.

max / min contre pmax / pmin — la distinction que tout le monde rate. « Note that `max` and `min` select **the largest and smallest values in their arguments, even if they are given several vectors**. The **parallel** maximum and minimum functions `pmax` and `pmin` return **a vector (of length equal to their longest argument) that contains in each element the largest (smallest) element in that position in any of the input vectors.** »

Énoncé. Soient `a <- c(1, 9, 3)` et `b <- c(7, 2, 5)`. Que valent `max(a, b)` et `pmax(a, b)` ?

Étape 1 — lire la définition de `max`. Elle sélectionne **la plus grande valeur parmi tous ses arguments**, tous vecteurs confondus. Elle **aplatit** donc les deux vecteurs en six nombres : 1, 9, 3, 7, 2, 5.

Étape 2 — le résultat. `max(a, b)` vaut **9** — un vecteur de **longueur 1**.

Étape 3 — lire la définition de `pmax`. Elle rend un vecteur **de la longueur du plus long argument**, contenant **en chaque position** le plus grand élément **de cette position**.

Étape 4 — dérouler position par position.

Position	a	b	pmax
1	1	7	7
2	9	2	9
3	3	5	5

Étape 5 — le résultat. `pmax(a, b)` vaut **c(7, 9, 5)** — un vecteur de **longueur 3**.

Étape 6 — le test de reconnaissance. « Le plus grand **du lot** » → `max`. « Le plus grand **de chaque paire** » → `pmax`. Un résultat de longueur 1 là où on en attendait n est la signature de l'erreur.

2.2 Entiers, réels, complexes

« For most purposes the user **will not be concerned** if the "numbers" in a numeric vector are integers, reals or even complex. **Internally calculations are done as double precision real numbers**, or double precision complex numbers if the input data are complex. » (§2.2)

Le nombre complexe doit être demandé (§2.2). « To work with complex numbers, **supply an explicit complex part**. »

```
sqrt(-17)      # NaN, avec un AVERTISSEMENT
sqrt(-17+0i)   # fait le calcul EN COMPLEXES
```

La règle générale est celle-ci : **R ne change pas de corps de nombres tout seul**. Si l'entrée est réelle, la sortie est réelle — quitte à valoir `NaN`. C'est vous qui déclarez l'intention, en écrivant `+0i`.

2.3 Le suffixe `L` — fabriquer un entier pour de bon

COMPLÉMENT (R LANGUAGE DEFINITION §3.1.1).

« Perhaps unexpectedly, **the number returned from the expression `1` is a numeric.** » Autrement dit, `1` n'est **pas** un entier : c'est un double.

« In most cases, the difference between an integer and a numeric value will be unimportant as R will do the right thing. There are, however, times when we would like to **explicitly create an integer value** for a constant. » Le moyen le plus simple est le **suffixe `L`** :

Écriture	Résultat
<code>1L</code>	l'entier 1
<code>0x10L</code>	l'entier 16 (hexadécimal)
<code>1e3L</code>	1000 comme entier — « Note that the <code>L</code> is treated as qualifying the term <code>1e3</code> and not the <code>3</code> », et c'est équivalent à <code>1000L</code>
<code>1e-3L</code>	avertissement , et c'est une valeur numérique qui est créée
<code>1.L</code>	avertissement : point décimal inutile
<code>12iL</code>	erreur de syntaxe : pas de <code>L</code> sur un complexe

● Concept 3 — Le recyclage : la règle qui décide de tout

DÉFINITION (§2.2).

« Vectors occurring in the same expression **need not all be of the same length**. If they are not, the value of the expression is a vector with **the same length as the longest vector** which occurs in the expression. **Recycling occurs in each binary operation**: the shorter vector is **recycled as often as need be (perhaps fractionally)** until it matches the length of the longer vector. **In particular a constant is simply repeated.** »

COMPLÉMENT — L'AVERTISSEMENT (R LANGUAGE DEFINITION §3.3.1).

« If one tries to add two structures with a different number of elements, then the shortest is recycled to length of longest. That is, if for instance you add `c(1, 2, 3)` to a six-element vector then you will really add `c(1, 2, 3, 1, 2, 3)`. **If the length of the longer vector is not a multiple of the shorter one, a warning is given.** »

Et : « As from **R 1.4.0**, any arithmetic operation involving a **zero-length vector** has a **zero-length result**. »

Les trois cas, et un seul est silencieux :

Situation	Comportement
Longueurs égales	élément par élément, rien à signaler
La longue est un multiple de la courte	recyclage silencieux — c'est le cas voulu
La longue n'est pas un multiple	recyclage fractionnaire + avertissement — le calcul se fait quand même
L'une est de longueur 0	résultat de longueur 0

Le troisième cas est le plus dangereux, précisément parce que le calcul aboutit. Un avertissement n'interrompt rien : vous obtenez un résultat de la bonne forme, faux au milieu. Dans un script non interactif, l'avertissement peut passer inaperçu.

Énoncé (celui du cours). Avec `x <- c(10.4, 5.6, 3.1, 6.4, 21.7)` (longueur 5) et `y <- c(x, 0, x)` (longueur 11), que produit `v <- 2*x + y + 1` ?

Le cours répond en une phrase : « generates a new vector `v` of **length 11** constructed by adding together, element by element, `2*x` **repeated 2.2 times**, `y` **repeated just once**, and `1` **repeated 11 times** ». Voici le déroulé complet.

Étape 1 — recenser les longueurs. `x` : 5. `y` : 11. La constante `1` : 1. Le plus long est `y`, donc **le résultat aura la longueur 11**.

Étape 2 — se rappeler que le recyclage est par opération binaire. La phrase du cours est « Recycling occurs in **each binary operation** ». L'expression contient trois opérations : `2*x`, puis `(2*x) + y`, puis `(...) + 1`. Chacune recycle **séparément**.

Étape 3 — la première opération, $2 \times x$. 2 est un vecteur de longueur 1, x de longueur 5. Le court est 2, répété 5 fois. Résultat de **longueur 5** : $c(20.8, 11.2, 6.2, 12.8, 43.4)$.

Étape 4 — la deuxième opération, $2 \times x + y$. Longueurs 5 et 11. **11 n'est pas un multiple de 5** : le recyclage est **fractionnaire** — d'où le « répété 2,2 fois » du cours — et **R émet un avertissement**. Le vecteur effectivement additionné à y est :

position	1	2	3	4	5	6	7	8	9	10	11
$2 \times x$ recyclé	20,8	11,2	6,2	12,8	43,4	20,8	11,2	6,2	12,8	43,4	20,8

Les positions 1 à 5 sont le premier passage, 6 à 10 le deuxième, et la position 11 est **le premier élément d'un troisième passage interrompu** : $11/5 = 2,2$.

Étape 5 — la troisième opération, $+ 1$. Longueurs 11 et 1 : la constante est **simplement répétée** 11 fois. Aucun avertissement — 11 est un multiple de 1.

Étape 6 — le résultat. Un vecteur de longueur 11, dont la 11^e valeur vaut $20,8 + y_{11} + 1 = 20,8 + 21,7 + 1 = 43,5$.

Étape 7 — l'interprétation, et l'avertissement à prendre au sérieux. Le cours donne cet exemple pour illustrer la **mécanique**, pas pour en recommander l'usage. Un recyclage fractionnaire est presque toujours **une erreur de conception** : on croit combiner deux séries alignées, et on aligne en réalité une série sur elle-même en décalé. **L'avertissement est ici le seul indice** — il ne faut pas l'ignorer.

● Concept 4 — Fabriquer des séquences

4.1 L'opérateur `:` et son piège de priorité

NOTATION (§2.3).

*« For example **1:30 is the vector** $c(1, 2, \dots, 29, 30)$. The colon operator has **high priority within an expression**, so, for example **2*1:15 is the vector** $c(2, 4, \dots, 28, 30)$. »*

Le piège que le cours vous fait découvrir vous-même. *« Put `n <- 10` and **compare the sequences** `1:n-1` and `1:(n-1)` . »*

Écriture	Ce que R lit	Résultat pour <code>n <- 10</code>
<code>1:n-1</code>	<code>(1:n) - 1</code> — le <code>:</code> est prioritaire	<code>0 1 2 3 4 5 6 7 8 9</code> — dix valeurs, commençant à 0
<code>1:(n-1)</code>	<code>1:9</code>	<code>1 2 3 4 5 6 7 8 9</code> — neuf valeurs, commençant à 1

Les deux vecteurs ont **des longueurs différentes** et **des premières valeurs différentes**. C'est la source d'erreur d'indice la plus fréquente chez le débutant, et elle est **silencieuse**.

« The construction `30:1` may be used to generate a sequence **backwards**. »

4.2 `seq()` — cinq arguments, jamais tous ensemble

DÉFINITION (§2.3).

« The function `seq()` is a **more general facility** for generating sequences. It has **five arguments, only some of which may be specified in any one call**. »

Argument	Rôle (§2.3)
<code>from=</code>	le début
<code>to=</code>	la fin
<code>by=</code>	le pas
<code>length=</code>	la longueur
<code>along=</code>	« normally used as the only argument , to create the sequence <code>1, 2, ..., Length(vector)</code> , or the empty sequence if the vector is empty (as it can be) »

Si seuls `from` et `to` sont donnés, **le résultat est le même que celui de l'opérateur deux-points** : `seq(2,10)` est le même vecteur que `2:10` .

MÉTHODE — LES ARGUMENTS NOMMÉS (§2.3).

« Arguments to `seq()` , **and to many other R functions**, can also be given in **named form**, in which case **l'ordre dans lequel ils apparaissent n'a aucune importance**. » Ainsi `seq(1,30)` , `seq(from=1, to=30)` et `seq(to=30, from=1)` **sont tous identiques à `1:30`** . Voir fiche 309 pour la règle générale d'appariement des arguments.

Les deux exemples du cours, et pourquoi ils donnent le même vecteur.

```
seq(-5, 5, by = .2) -> s3          # de -5 a 5 par pas de 0,2
s4 <- seq(length = 51, from = -5, by = .2)
```

« generates in `s3` the vector `c(-5.0, -4.8, -4.6, ..., 4.6, 4.8, 5.0)` », et `s4` **le même vecteur**.
Vérification : de **-5** à **5** par pas de **0,2**, le nombre de valeurs est $\frac{5 - (-5)}{0,2} + 1 = 50 + 1 = 51$. Les deux écritures spécifient donc bien la même chose — l'une par la **borne**, l'autre par le **compte**.

`along=` **est la seule protection contre le vecteur vide**. `1:length(v)` sur un `v` vide donne `1 0` — c'est-à-dire `c(1, 0)` , **deux itérations** au lieu de zéro. `seq(along = v)` donne **la séquence vide**, exactement comme il faut. C'est la raison d'être de cet argument, et le cours le souligne : « *or the empty sequence if the vector is empty (as it can be)* ».

4.3 `rep()` — deux façons de répéter

DÉFINITION (§2.3).

« A related function is `rep()` which can be used for **replicating an object in various complicated ways**. »

```
s5 <- rep(x, times = 5)    # « cinq copies de x bout a bout »
s6 <- rep(x, each = 5)     # « repete CHAQUE element de x cinq fois avant de passer au suivant »
```

Pour <code>x <- c(1, 2, 3)</code>	Résultat
<code>rep(x, times = 3)</code>	<code>1 2 3 1 2 3 1 2 3</code> — le motif se répète
<code>rep(x, each = 3)</code>	<code>1 1 1 2 2 2 3 3 3</code> — chaque valeur se répète

Les deux ont la même longueur et un ordre entièrement différent. `times` sert à **répliquer une structure** ; `each` sert à **fabriquer des groupes** — d'où son rôle central dans la construction des facteurs (fiche 304).

● Concept 5 — Les vecteurs logiques

DÉFINITION (§2.4).

« As well as numerical vectors, R allows manipulation of **logical quantities**. The elements of a logical vector can have the values `TRUE`, `FALSE`, and `NA` (for "not available"). »

« **Logical vectors are generated by conditions.** »

```
temp <- x > 13
```

*met dans `temp` un vecteur **de la même longueur que** `x`, valant `FALSE` là où la condition n'est pas remplie et `TRUE` là où elle l'est.*

Famille (§2.4)	Opérateurs
Comparaison	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> pour l'égalité exacte <code>!=</code> pour l'inégalité
Logique	<code>c1 & c2</code> intersection (« et ») · <code>c1 c2</code> union (« ou ») · <code>!c1</code> négation

5.1 Le piège T et F

Le piège (§2.4), déjà annoncé en fiche 300. « *The first two are often abbreviated as `T` and `F`, respectively. Note however that `T` and `F` are just variables which are set to `TRUE` and `FALSE` by default, but are not reserved words and hence can be overwritten by the user. Hence, you should always use `TRUE` and `FALSE`.* »

La démonstration tient en trois lignes :

```
T           # TRUE
T <- 0      # parfaitement legal : T n'est pas un mot reserve
if (1 == 1 && T) "ok" else "casse"  # « casse »
```

Rien n'a protesté. Toute expression écrite avec `T` ailleurs dans la session est désormais fausse, **silencieusement**.

5.2 La coercition logique → numérique

Règle (§2.4). « *Logical vectors may be used in **ordinary arithmetic**, in which case they are **coerced into numeric vectors**, `FALSE` becoming 0 and `TRUE` becoming 1.* »

C'est ce qui rend l'idiome suivant possible — et c'est l'un des plus utiles de R :

```
sum(x > 13)      # COMBIEN d'elements dépassent 13 : on somme des 1
mean(x > 13)     # quelle PROPORTION : la moyenne de 0 et de 1
```

Mais la coercition n'est pas une équivalence (§2.4). « *However there are situations where **logical vectors and their coerced numeric counterparts are not equivalent**, for example see the next subsection.* » Le cours renvoie explicitement aux **valeurs manquantes**, et surtout à l'**indexation** : un vecteur logique et un vecteur de 0/1 **n'indexent pas de la même façon** (fiche 302). `x[c(TRUE, FALSE)]` prend un élément sur deux ; `x[c(1, 0)]` prend le premier élément, une fois.

● Concept 6 — Valeurs manquantes : NA et NaN

DÉFINITION (§2.5).

*« When an element or value is **"not available"** or a **"missing value"** in the statistical sense, a place within a vector may be reserved for it by assigning it **the special value NA**. In general any operation on an NA becomes an NA . »*

INTUITION — LA JUSTIFICATION DU COURS, QUI VAUT D'ÊTRE RETENUE (§2.5).

« The motivation for this rule is simply that **if the specification of an operation is incomplete, the result cannot be known and hence is not available**. » NA n'est pas « zéro », ni « rien » : c'est « je ne sais pas ». Et tout calcul fait à partir de quelque chose qu'on ne sait pas donne quelque chose qu'on ne sait pas.

6.1 Le piège central : `x == NA` ne teste rien

PROBLÈME (§2.5).

« Notice that the logical expression `x == NA` is quite different from `is.na(x)` since NA is not really a value but a marker for a quantity that is not available. Thus `x == NA` is a vector of the same length as `x` all of whose values are NA , as the logical expression itself is incomplete and hence undecidable. »

```
z <- c(1:3, NA)
ind <- is.na(z)      # FALSE FALSE FALSE TRUE    <- LE bon test
z == NA             # NA NA NA NA              <- ne teste rien
```

Pourquoi `1 == NA` vaut NA et non FALSE : la question posée est « 1 est-il égal à une valeur que je ne connais pas ? ». La réponse honnête n'est pas « non », c'est « **je ne peux pas savoir** ». R répond NA .

« The function `is.na(x)` gives a logical vector of the same size as `x` with value TRUE **if and only if** the corresponding element in `x` is NA . »

6.2 NaN — l'autre sorte de manquant

DÉFINITION (§2.5).

*« Note that there is a **second kind of "missing" values** which are produced by **numerical computation**, the so-called **Not a Number**, `NaN`, values. »*

`0/0` `# NaN`

`Inf - Inf` `# NaN`

« which both give `NaN` since **the result cannot be defined sensibly** ».

	NA	NaN
Origine	une donnée absente	un calcul indéfini
Sens	« non disponible »	« pas un nombre »
<code>is.na()</code>	<code>TRUE</code>	<code>TRUE</code> aussi
<code>is.nan()</code>	<code>FALSE</code>	<code>TRUE</code>

Le résumé du cours (§2.5). « In summary, `is.na(xx)` is `TRUE` both for `NA` and `NaN` values. To differentiate these, `is.nan(xx)` is only `TRUE` for `NaN` s. »

« Missing values are sometimes printed as `<NA>` when character vectors are printed **without quotes**. »

6.3 Les types de `NA` — le complément décisif de la *R Language Definition*

Cette partie n'est pas dans *An Introduction to R*. Elle est dans §3.3.4 de la *R Language Definition*, et elle explique des comportements autrement inexplicables.

DÉFINITION.

« As the elements of an **atomic vector must be of the same type**, there are **multiple types of NA values**. There is one case where this is particularly important to the user : **the default type of NA is logical**, unless coerced to some other type, so **the appearance of a missing value may trigger logical rather than numeric indexing**. »

Conséquence immédiate, que le manuel énonce dans sa section indexation (fiche 302) : `x[NA]` a la longueur de `x`, tandis que `x[c(1, NA)]` a la longueur 2 — dans le premier cas ce sont les règles de l'index **logique** qui s'appliquent, dans le second celles de l'index **entier**.

FORMULE — LES CAS OÙ NA NE SE PROPAGE PAS.

« Numeric and logical calculations with NA **generally return NA**. In cases where the result of the operation would be the same for all possible values the NA could take, the operation may return this value. In particular, `FALSE & NA` is `FALSE`, `TRUE | NA` is `TRUE`. »

Expression	Résultat	Pourquoi
<code>FALSE & NA</code>	<code>FALSE</code>	un « et » avec un <code>FALSE</code> est faux quelle que soit l'autre valeur
<code>TRUE NA</code>	<code>TRUE</code>	un « ou » avec un <code>TRUE</code> est vrai quelle que soit l'autre valeur
<code>TRUE & NA</code>	<code>NA</code>	là, la réponse dépendrait de la valeur inconnue
<code>NA == NA</code>	<code>NA</code>	« <code>NA</code> is not equal to any other value or to itself »

Mais `match` fait exception : « However, an NA value **will match another NA value in match** ». Donc `NA %in% c(1, NA)` vaut `TRUE`, alors que `NA == NA` vaut `NA`. Deux notions différentes : **égalité** (indécidable) et **appariement** (une opération de recherche, qui traite NA comme un motif).

Les autres règles de §3.3.4, à connaître :

- `NaN` « exists **only in the double type** and for real or imaginary components of the complex type ».
- **Coercer** `NaN` en logique ou en entier donne un `NA` du type approprié ; en caractère, cela donne la chaîne `"NaN"`.

- *« `NaN` values are **incomparable**, so tests of equality or collation involving `NaN` will result in `NA`. They are regarded as **matching any `NaN` value** (and no other value, **not even `NA`**) by `match`. »*
- *« The **`NA` of character type is distinct from the string `"NA"`**. Programmers who need to specify an explicit string `NA` should use `NA_character_` rather than `"NA"`, or set elements to `NA` using `is.na<-`. »*
- Les constantes `NA_integer_`, `NA_real_`, `NA_complex_` et `NA_character_` « *will generate (in the parser) an `NA` value of the appropriate type, and will be used **in deparsing** when it is not otherwise possible to identify the type of an `NA`* ».
- « There is **no `NA` value for raw vectors.** »

Ne pas confondre `NA` et l'argument manquant. « This should **not be confused with the missing property for a function argument that has not been supplied** » (§3.3.4, qui renvoie à §4.1.2). Un argument non fourni n'est pas `NA` : il n'existe pas. Voir fiche 309.

Énoncé. `z <- c(1:3, NA)`. Que vaut `mean(z)` ? Comment obtenir la moyenne des valeurs connues ?

Étape 1 — appliquer la règle. « In general **any operation on an `NA` becomes an `NA`** ». La somme `1 + 2 + 3 + NA` vaut `NA`, donc la moyenne aussi. `mean(z)` vaut `NA`.

Étape 2 — comprendre que ce n'est pas un bug. C'est la réponse correcte à la question posée. La moyenne de quatre nombres dont un est inconnu **est** inconnue. Si R répondait `2`, il répondrait à une **autre** question.

Étape 3 — poser la bonne question, voie 1 : filtrer. La technique du §2.7 :

```
z[!is.na(z)]      # 1 2 3
mean(z[!is.na(z)]) # 2
```

Ne pas écrire `z[z != NA]` : `z != NA` vaut `NA NA NA NA`, et indexer par des `NA` renvoie des `NA`.

Étape 4 — poser la bonne question, voie 2 : l'argument dédié.

```
mean(z, na.rm = TRUE) # 2
```

Étape 5 — laquelle choisir. `na.rm = TRUE` est plus lisible et évite de recopier le vecteur ; le filtrage explicite est nécessaire quand **plusieurs** vecteurs doivent rester alignés (garder les mêmes positions dans `x` et dans `y`).

Étape 6 — vérifier le dénominateur. `mean(z, na.rm = TRUE)` divise par **3**, pas par 4 : `sum = 6` , `n = 3` , moyenne **2**. Les valeurs manquantes sont **retirées**, elles ne sont pas remplacées par zéro. Si elles l'étaient, on obtiendrait $6/4 = 1,5$ — c'est l'erreur que produit un `z[is.na(z)] <- 0` fait sans réfléchir.

Enrichissement pédagogique (hors cours) : l'argument `na.rm` n'est pas décrit au §2.5 d'An Introduction to R ; il est documenté dans les pages d'aide de `mean` , `sum` , `var` (Reference Index, paquet `base`). Le filtrage par `!is.na()` , lui, est bien la technique donnée par le cours au §2.7.

● Concept 7 — Les vecteurs de caractères

DÉFINITION (§2.6).

« Character quantities and character vectors are used frequently in R, for example **as plot labels**. Where needed they are denoted by a **sequence of characters delimited by the double quote character**, e.g. `"x-values"` , `"New iteration results"` . »

Les guillemets (§2.6). « Character strings are entered using **either matching double (`"`) or single (`'`) quotes**, but are **printed using double quotes** (or sometimes without quotes). »

Les échappements (§2.6). « They use **C-style escape sequences**, using `\` as the escape character, so `\` is entered and printed as `\\` , and inside double quotes `"` is entered as `\"` . Other useful escape sequences are `\n` , newline, `\t` , tab and `\b` , backspace — see `?Quotes` for a full list. »

COMPLÉMENT — LA LISTE DES ÉCHAPPEMENTS (R LANGUAGE DEFINITION §10.3.1).

Au-delà de `\n`, `\t`, `\b` et `\\` : `\nnn` (code **octal**, une à trois chiffres de 0 à 7), `\xnn` (code **hexadécimal**, un ou deux chiffres), `\unnnn` ou `\u{nnnn}` (Unicode, jusqu'à **quatre** chiffres hexadécimaux), `\Unnnnnnnn` ou `\U{nnnnnnnn}` (Unicode, jusqu'à **huit**). Les formes `\u` et `\U` exigent un environnement multi-octets ; « *otherwise an error* », et le caractère « *needs to be valid in the current locale* ».

« A **NUL (`\0`) is not allowed** in a character string, so using `\0` in a string constant **terminates the constant** (usually with a warning) : further characters up to the closing quote are scanned but ignored. »

Et une commodité : « A single quote may also be **embedded directly** in a double-quote delimited string **and vice versa**. » — la règle déjà vue en fiche 300 avec `"It's important"`.

7.1 `paste()` — et le recyclage qui frappe encore

DÉFINITION (§2.6).

*« The `paste()` function takes an **arbitrary number of arguments** and **concatenates them one by one into character strings**. Any numbers given among the arguments are coerced into **character strings** in the evident way — that is, in the same way they would be if they were printed. The arguments are **by default separated in the result by a single blank character**, but this can be changed by the named argument `sep=string`, possibly empty. »*

Énoncé (celui du cours, §2.6).

```
labs <- paste(c("X","Y"), 1:10, sep = "")
```

Que contient `labs` ?

Étape 1 — recenser les longueurs. Premier argument : **2**. Deuxième : **10**. Le résultat aura la **longueur 10**.

Étape 2 — appliquer le recyclage. Le cours l'écrit noir sur blanc : « Note particularly that **recycling of short lists takes place here too** ; thus `c("X", "Y")` is **repeated 5 times** to match the sequence

`1:10` . » 10 est un multiple de 2 : **aucun avertissement**.

Étape 3 — dérouler l'appariement.

position	1	2	3	4	5	6	7	8	9	10
lettre recyclée	X	Y	X	Y	X	Y	X	Y	X	Y
nombre	1	2	3	4	5	6	7	8	9	10

Étape 4 — appliquer `sep = ""` . Le séparateur par défaut est **une espace** ; ici il est **vide**, donc lettre et nombre sont collés.

Étape 5 — le résultat, tel que le cours le donne.

```
c("X1", "Y2", "X3", "Y4", "X5", "Y6", "X7", "Y8", "X9", "Y10")
```

Étape 6 — pourquoi c'est surprenant, et ce qu'on voulait sans doute. On attend souvent « X1...X10 puis Y1...Y10 », soit **20** étiquettes. On en obtient **10**, alternées. Pour l'autre résultat, il faut faire varier les deux facteurs explicitement — par exemple avec `rep(..., each = 10)` , ce qui ramène au concept 4 :

```
paste(rep(c("X","Y"), each = 10), 1:10, sep = "")
# "X1" ... "X10" "Y1" ... "Y10" -- 20 étiquettes
```

Sans avertissement, un recyclage reste un recyclage. Le silence de R signifie « les longueurs sont compatibles », pas « c'est ce que tu voulais ».

COMPLÉMENT — COLLAPSE (NOTE 3, §2.6).

« `paste(..., collapse=ss)` **joins the arguments into a single character string** putting `ss` in between, e.g. `ss <- "/"` . » Deux arguments à ne pas confondre : `sep` sépare les **colonnes** (et le résultat garde sa longueur) ; `collapse` réduit le vecteur à **un seul élément**.

« There are **more tools for character manipulation**, see the help for `sub` and `substring` . »

● Concept 8 — La table des opérateurs

NOTATION (R LANGUAGE DEFINITION §3.1.4).

« R contains a number of operators. They are listed in the table below. »

Opérateur	Rôle
-	Moins , peut être unaire ou binaire
+	Plus , peut être unaire ou binaire
!	Non unaire
~	Tilde , pour les formules de modèle ; peut être unaire ou binaire
?	Aide
:	Séquence , binaire (<i>dans une formule de modèle</i> : interaction)
* / ^	multiplication, division, exponentiation , binaires
%x%	opérateurs binaires spéciaux — « <code>x</code> can be replaced by any valid name »
%%	modulo , binaire
%%/	division entière , binaire
%%*	produit matriciel , binaire
%o%	produit extérieur , binaire
%x%	produit de Kronecker , binaire
%in%	opérateur d'appariement , binaire (<i>dans une formule</i> : imbrication)
% %	opérateur de coalescence nulle , binaire
< > == >= <=	inférieur, supérieur, égal, supérieur ou égal, inférieur ou égal — binaires
&	et , binaire, vectorisé
&&	et , binaire, non vectorisé
	ou , binaire, vectorisé
	ou , binaire, non vectorisé
<- / ->	assignation à gauche / à droite , binaires
\$	sous-ensemble de liste , binaire

Deux remarques de lecture.

1. `!=` ne figure pas dans cette table de la *R Language Definition*. Il existe bel et bien, et *An Introduction to R* le donne au §2.4 (« `!=` for inequality »). C'est une lacune de la table, pas du langage.
2. `&` contre `&&` est la distinction la plus lourde de conséquences du tableau : `&` travaille **élément par élément** et rend un vecteur ; `&&` rend **une seule valeur** et sert aux conditions de `if`. Voir fiche 308.

Ce qu'il faut retenir de plus profond (§3.1.4). *« Except for the syntax, there is no difference between applying an operator and calling a function. In fact, `x + y` can equivalently be written ``+`(x, y)`. Notice that since `+` is a **non-standard function name**, it needs to be quoted. »*

De même pour l'indexation : « Like the other operators, **indexing is really done by functions**, and one could have used ``[(x, 2)` instead of `x[2]`. » Cette identité opérateur = fonction est le fondement de la fiche 317 (calcul sur le langage) — et elle explique pourquoi on peut écrire `sapply(liste, [[, 1)`.

`%x%` apparaît deux fois dans la table : une fois comme **forme générale** des opérateurs binaires définis par l'utilisateur (« `x` peut être remplacé par n'importe quel nom valide »), une fois comme l'opérateur **concret** du produit de Kronecker. Ce n'est pas une coquille : c'est la même syntaxe, employée à deux niveaux. Voir fiche 309 pour définir le vôtre.

● Concept 9 — La carte du reste : les autres types d'objets

DÉFINITION (§2.8).

« **Vectors are the most important type of object in R**, but there are several others which we will meet more formally in later sections. »

Objet (§2.8)	Ce que le cours en dit	Fiche
matrices , plus généralement arrays	« <i>multi-dimensional generalizations of vectors. In fact, they are vectors that can be indexed by two or more indices and will be printed in special ways</i> »	305
factors	« <i>provide compact ways to handle categorical data</i> »	304
lists	« <i>a general form of vector in which the various elements need not be of the same type, and are often themselves vectors or lists. Lists provide a convenient way to return the results of a statistical computation</i> »	306
data frames	« <i>matrix-like structures, in which the columns can be of different types. Think of data frames as "data matrices" with one row per observational unit but with (possibly) both numerical and categorical variables</i> »	306
functions	« <i>are themselves objects in R which can be stored in the project's workspace. This provides a simple and convenient way to extend R</i> »	309

La phrase à garder. « ***In fact, they are vectors** that can be indexed by two or more indices.* » Une matrice n'est **pas** un nouveau type : c'est **un vecteur muni d'un attribut `dim`**. C'est exactement ce que démontre la fiche 303, et ce qui explique que `m[1]` — un seul indice sur une matrice — rende toujours quelque chose de sensé.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
deux vecteurs de longueurs différentes dans un calcul	recyclage : longueur du résultat = celle du plus long ; avertissement si non-multiple
« pourquoi ai-je un avertissement <code>longer object length is not a multiple of 4</code> ? »	recyclage fractionnaire — presque toujours une erreur de conception
« le résultat est vide »	un opérande de longueur 0 → résultat de longueur 0
« combien d'éléments vérifient... ? »	<code>sum(condition)</code> — coercion <code>TRUE</code> → 1
« quelle proportion... ? »	<code>mean(condition)</code>
« le plus grand de chaque paire »	<code>pmax</code> , pas <code>max</code>
« ma moyenne vaut <code>NA</code> »	propagation du <code>NA</code> → <code>na.rm = TRUE</code> ou <code>x[!is.na(x)]</code>
« mon test <code>== NA</code> ne marche pas »	il ne peut pas marcher → <code>is.na()</code>
« distinguer une donnée absente d'un calcul raté »	<code>is.na()</code> vaut <code>TRUE</code> pour les deux ; <code>is.nan()</code> isole <code>NaN</code>
« ma boucle tourne une fois de trop sur un vecteur vide »	<code>1:length(v)</code> → <code>seq(along = v)</code>
« mon indice commence à 0 »	<code>1:n-1</code> lu comme <code>(1:n)-1</code> → parenthéser
une racine de nombre négatif	<code>sqrt(-17)</code> = <code>NaN</code> ; il faut <code>sqrt(-17+0i)</code>
construire des étiquettes	<code>paste()</code> + <code>sep</code> ; <code>collapse</code> pour tout réduire à une chaîne
fabriquer des groupes égaux	<code>rep(x, each = k)</code> ; répéter un motif : <code>times</code>
un <code>if</code> qui proteste sur une condition de longueur > 1	<code>&</code> / <code> </code> sont vectorisés , <code>&&</code> / <code> </code> non

Comment résoudre ce type d'exercice

Protocole « prévoir le résultat d'une expression vectorielle » — 5 étapes.

1. **Noter la longueur de chaque opérande**, la constante comprise (longueur 1).

2. **Décomposer en opérations binaires** — le recyclage se fait à **chaque opération**, pas une fois pour l'expression.
3. Pour chaque opération : longueur du résultat = **le maximum** ; vérifier si le max est **un multiple** du min (sinon, **avertissement**).
4. **Dérouler le recyclage position par position** dans un tableau, au moins pour la première opération douteuse.
5. **Vérifier le type** du résultat : un logique entré dans une opération arithmétique **devient 0/1** ; un `NA` quelque part **se propage**, sauf `FALSE & NA` et `TRUE | NA` .

Protocole « nettoyer un vecteur de ses manquants » — 4 étapes.

1. **Diagnostiquer** : `is.na(x)` pour l'ensemble manquant, `is.nan(x)` pour distinguer les calculs indéfinis.
2. **Décider** : retirer (`na.rm = TRUE` ou `x[!is.na(x)]`) ou remplacer (`x[is.na(x)] <- 0`) — les deux **ne donnent pas le même dénominateur**.
3. **Ne jamais tester par ==** : `x == NA` rend `NA` partout.
4. **Vérifier la longueur** après coup : filtrer **raccourcit** le vecteur, ce qui **désaligne** les vecteurs parallèles.

Protocole « fabriquer la bonne séquence » — 4 étapes.

1. Bornes connues, pas de 1 → `a:b` ; se méfier de la **priorité** du `:` , parenthéser au moindre doute.
2. Pas différent de 1 → `seq(from, to, by =)` ; nombre de valeurs voulu → `seq(length =, from =, by =)` .
3. Indexer un vecteur existant → `seq(along = v)` , jamais `1:length(v)` .
4. Répéter → `times` pour le motif, `each` pour les groupes.

● Common mistakes

Erreur	Correction
Tester <code>x == NA</code>	impossible par construction → <code>is.na(x)</code>
Croire <code>is.na()</code> spécifique à <code>NA</code>	il vaut <code>TRUE</code> pour <code>NaN</code> aussi → <code>is.nan()</code> pour distinguer
Croire <code>NA == NA</code> égal à <code>TRUE</code>	c'est <code>NA</code> — mais <code>match / %in%</code> apparie les <code>NA</code>
Croire que tout calcul avec <code>NA</code> donne <code>NA</code>	<code>FALSE & NA</code> vaut <code>FALSE</code> , <code>TRUE NA</code> vaut <code>TRUE</code>
Confondre le <code>NA</code> caractère et la chaîne <code>"NA"</code>	ce sont deux choses distinctes → <code>NA_character_</code>
Confondre <code>NA</code> et un argument non fourni	l'un est une valeur , l'autre une absence d'argument (<code>missing</code>)
Ignorer l'avertissement de recyclage	le calcul aboutit quand même , faux au milieu
Croire que le recyclage s'applique une fois à l'expression	il s'applique à chaque opération binaire
Écrire <code>1:n-1</code> pour « de 1 à n-1 »	le <code>:</code> est prioritaire → <code>1:(n-1)</code>
Écrire <code>1:length(v)</code>	sur un <code>v</code> vide , cela vaut <code>c(1, 0)</code> → <code>seq(along = v)</code>
Utiliser <code>max</code> pour un maximum position par position	c'est <code>pmax</code>
Attendre un complexe de <code>sqrt(-17)</code>	il faut <code>sqrt(-17+0i)</code>
Utiliser <code>T</code> et <code>F</code>	variables réassignables → <code>TRUE</code> et <code>FALSE</code>
Confondre <code>rep(x, times=k)</code> et <code>rep(x, each=k)</code>	motif répété contre valeurs groupées
Attendre 20 étiquettes de <code>paste(c("X", "Y"), 1:10)</code>	recyclage : 10 étiquettes alternées
Confondre <code>sep</code> et <code>collapse</code>	<code>sep</code> sépare dans chaque élément ; <code>collapse</code> réduit à un seul élément
Utiliser <code>&&</code> sur des vecteurs	<code>&&</code> est non vectorisé — pour un vecteur, c'est <code>&</code>
Écrire <code>1</code> en croyant écrire un entier	<code>1</code> est un double ; l'entier est <code>1L</code>

Erreur	Correction
Écrire <code>1e-3L</code> en attendant un entier	avertissement , et c'est un numérique qui est créé
Croire qu'un <code>NA</code> dans un index se comporte toujours pareil	<code>x[NA]</code> suit les règles logiques , <code>x[c(1, NA)]</code> les règles entières
Croire <code>c()</code> capable d'imbriquer	<code>c(x, 0, x)</code> aplatit — 11 éléments, pas 3

Ultimate Review

Le vecteur. *La structure la plus simple ; une **collection ordonnée**.* Un nombre isolé est un vecteur de longueur 1. `c()` **concatène bout à bout** — donc **aplatit**. Quatre écritures d'assignation : `<-`, `=` (« dans la plupart des contextes »), `assign("x", ...)`, `-> x`.

La vectorisation. Toute opération est **élément par élément**, avec la **boucle implicite écrite en C**. `x + y` s'écrit aussi ``+`(x, y)` : « *except for the syntax, there is no difference between applying an operator and calling a function* ».

Le recyclage — la règle en quatre lignes. Longueur du résultat = **celle du plus long** · le plus court est **répété, éventuellement fractionnairement** · **non-multiple** → **avertissement** · **longueur 0 quelque part** → **résultat de longueur 0** (depuis R 1.4.0). Et il s'applique à **chaque opération binaire**, pas une fois pour l'expression.

Les fonctions de base. `length` `sum` `prod` `max` `min` `range` = `c(min, max)` · `mean` = `sum/length` · `var` = $\frac{1}{n-1} \sum (x_i - \bar{x})^2$, et **matrice de covariance** $p \times p$ si l'argument est une matrice $n \times p$ · `sort` trie, `order` / `sort.list` **rendent la permutation** · `pmax` / `pmin` **parallèles**.

Séquences. `1:30` · le `:` **est prioritaire** : `2*1:15` = `c(2,...,30)`, et `1:n-1` = `(1:n)-1` · `30:1` à rebours · `seq()` a **cinq** arguments (`from`, `to`, `by`, `length`, `along`), jamais tous ensemble · `seq(2,10)` = `2:10` · `along=` **protège du vecteur vide** · `rep(x, times=)` répète le **motif**, `rep(x, each=)` répète **chaque valeur**.

Logiques. Valeurs `TRUE`, `FALSE`, `NA` · générés **par des conditions** · `<` `<=` `>` `>=` `==` `!=` · `&` `|` `!` · **coercition en arithmétique** : `FALSE` → `0`, `TRUE` → `1`, d'où `sum(cond)` et `mean(cond)` · `T` et `F` **sont des variables, pas des mots réservés**.

Manquants. `NA` = donnée non disponible ; `NaN` = calcul indéfini (`0/0`, `Inf - Inf`) · `is.na()` **vaut TRUE pour les deux**, `is.nan()` **pour NaN seul** · `x == NA` **rend NA partout — ce n'est pas un test** · `NA` n'est égal ni à rien ni à lui-même, **mais** `match` **apparie les NA** · `FALSE & NA` = `FALSE`, `TRUE | NA` = `TRUE` · le **type par défaut de NA est logique**, d'où `x[NA]` de la longueur de `x` et `x[c(1,NA)]` de

longueur 2 · `NA_character_` ≠ `"NA"` · **pas de NA pour les vecteurs** `raw` · affichage `<NA>` sans guillemets.

Caractères. Guillemets **doubles ou simples, affichés en doubles** · échappements à la C : `\\` `\"` `\n` `\t` `\b`, plus `\nnn` (octal), `\xnn` (hexa), `\unnnn` / `\Unnnnnnnn` (Unicode) · `\0` **interdit** : il termine la constante · `paste()` **coerce les nombres**, sépare par **une espace** par défaut, `sep=` change, `collapse=` **réduit à une seule chaîne** · le recyclage s'y applique aussi.

Constantes. `1` est un **double** ; `1L` un **entier** · `0x10L` = 16 · `1e3L` = `1000L` (le `L` qualifie `1e3`) · `1e-3L` et `1.L` → **avertissement** · `12iL` → **erreur**.

Opérateurs. `%%` modulo · `/%%` division entière · `%*%` produit matriciel · `%o%` produit extérieur · `%x%` Kronecker · `%in%` appariement · `%|%` coalescence nulle · `& / |` **vectorisés**, `&& / ||` **non** · `%nom%` pour définir le sien · `!=` **manque à la table de la R Language Definition**, mais existe.

Les cinq autres objets (§2.8). matrices/arrays (**des vecteurs à plusieurs indices**), facteurs (catégoriel compact), listes (**éléments de types différents**), data frames (**colonnes de types différents, une ligne par unité d'observation**), fonctions (**des objets**, stockables dans l'espace de travail).

Active Recall

*« If they are not [of the same length], the value of the expression is a vector with **the same length as the longest** vector which occurs in the expression. **Recycling occurs in each binary operation**: the shorter vector is **recycled as often as need be (perhaps fractionally)**. In particular a constant is simply repeated. »* (§2.2)

Quand elle tombe mal (R Language Definition §3.3.1) : « If the length of the longer vector is **not a multiple** of the shorter one, **a warning is given** » — mais **le calcul se fait quand même**. Et : « any arithmetic operation involving a **zero-length vector** has a **zero-length result** » (depuis R 1.4.0).

Trois opérations binaires, chacune recyclant séparément.

1. `2*x` : longueurs 1 et 5 → résultat de **longueur 5**, la constante répétée.
2. `2*x + y` : longueurs 5 et 11. **11 n'est pas un multiple de 5** → recyclage **fractionnaire** (« `2*x` repeated **2,2 times** ») et **avertissement**. La 11^e position reprend le **1^{er}** élément de `2*x`.

3. `... + 1` : longueurs 11 et 1 → la constante est répétée **11 fois**, sans avertissement.

Résultat : un vecteur de **longueur 11**. Et le message essentiel : un recyclage fractionnaire est presque toujours **une erreur**, dont **l'avertissement est le seul indice**.

*« `NA` is not really a value but a marker for a quantity that is not available. Thus `x == NA` is a vector of the same length as `x` all of whose values are `NA`, as the logical expression itself is **incomplete and hence undecidable**. »* (§2.5)

La question « 1 est-il égal à une valeur inconnue ? » n'a pas pour réponse « non » mais « **je ne peux pas savoir** ». Le bon test est `is.na(x)`, qui rend `TRUE` **si et seulement si** l'élément correspondant est `NA`.

	NA	NaN
Origine	une donnée absente	un calcul indéfini (<code>0/0</code> , <code>Inf - Inf</code>)
<code>is.na()</code>	<code>TRUE</code>	<code>TRUE</code> aussi
<code>is.nan()</code>	<code>FALSE</code>	<code>TRUE</code>

« In summary, `is.na(xx)` is `TRUE` **both for** `NA` **and** `NaN` **values**. To differentiate these, `is.nan(xx)` is **only** `TRUE` **for** `NaN` **s**. » (§2.5)

`NaN` « exists **only in the double type** » et pour les composantes d'un complexe.
Coercé en logique ou en entier il donne un `NA` ; en caractère, la chaîne `"NaN"`.

*« In cases where **the result of the operation would be the same for all possible values the** `NA` **could take**, the operation may return this value. In particular, `FALSE & NA` is `FALSE`, `TRUE | NA` is `TRUE`. »* (R Language Definition §3.3.4)

La logique est celle de l'**évaluation suffisante** : un « et » avec un `FALSE` est faux quoi qu'il arrive ; un « ou » avec un `TRUE` est vrai quoi qu'il arrive. En revanche `TRUE & NA` vaut `NA`.

— là, la réponse **dépendrait** de la valeur inconnue.

Autre exception, d'une autre nature : « *an NA value will match another NA value in match* », alors que `NA == NA` vaut `NA`. **Égalité** et **appariement** ne sont pas la même opération.

« *The colon operator has **high priority** within an expression* » (§2.3).

- `1:n-1` est lu `(1:10) - 1` → `0 1 2 ... 9` : **dix** valeurs, commençant à **0**.
- `1:(n-1)` est `1:9` → `1 2 ... 9` : **neuf** valeurs, commençant à **1**.

Longueur différente **et** première valeur différente. C'est le cours lui-même qui invite à la comparaison : « *Put `n <- 10` and compare the sequences* ». Même mécanisme pour `2*1:15`, qui vaut `c(2, 4, ..., 30)` et non `(2*1):15`.

Parce que `along=` produit « *the sequence 1, 2, ..., Length(vector), or the empty sequence if the vector is empty (as it can be)* » (§2.3).

Sur un vecteur vide, `length(v)` vaut `0` et `1:0` vaut `c(1, 0)` — soit **deux** itérations là où il en fallait **zéro**, avec deux indices dont l'un est hors bornes. `seq(along = v)` rend la séquence vide. Le cas du vecteur vide est rare en interactif et fréquent en production : c'est précisément le genre de bug qui n'apparaît qu'une fois le code livré.

`max / pmax` : « *max and min select the largest and smallest values in their arguments, even if they are given several vectors* » — résultat de **longueur 1**. « *The parallel maximum and minimum functions `pmax` and `pmin` return a vector (of length equal to their longest argument) that contains in each element the largest element in that position* ». Pour `a = c(1,9,3)`, `b = c(7,2,5)` : `max` → **9** ; `pmax` → `c(7, 9, 5)`.

`times / each` : pour `x = c(1,2,3)`, `rep(x, times=3)` → `1 2 3 1 2 3 1 2 3` (le **motif** se répète) ; `rep(x, each=3)` → `1 1 1 2 2 2 3 3 3` (chaque **valeur** se répète). Même longueur, ordre entièrement différent. `each` sert à fabriquer des **groupes**.

```
c("X1", "Y2", "X3", "Y4", "X5", "Y6", "X7", "Y8", "X9", "Y10")
```

Dix étiquettes, alternées. « *Note particularly that **recycling of short lists takes place here too** ; thus `c("X", "Y")` is **repeated 5 times** to match the sequence `1:10` .* » (§2.6) Le résultat a la longueur du **plus long** argument, soit 10 ; et 10 étant un multiple de 2, **aucun avertissement** ne le signale.

Pour les vingt étiquettes attendues, il faut faire varier les deux facteurs :

```
paste(rep(c("X","Y"), each = 10), 1:10, sep = "")
```

```
sum(x > 13)      # le NOMBRE  
mean(x > 13)     # la PROPORTION
```

Cela repose sur la coercion du §2.4 : « *Logical vectors may be used in ordinary arithmetic, in which case they are **coerced into numeric vectors**, `FALSE` becoming 0 and `TRUE` becoming 1* ». Sommer des `TRUE` , c'est les compter ; en faire la moyenne, c'est en prendre la proportion.

Le cours prévient aussitôt : « ***there are situations where logical vectors and their coerced numeric counterparts are not equivalent*** ». En **indexation** notamment :

`x[c(TRUE, FALSE)]` prend un élément sur deux, `x[c(1, 0)]` prend le premier une fois (fiche 302).

Rien — et c'est le problème. « *`T` and `F` are **just variables** which are set to `TRUE` and `FALSE` by default, but **are not reserved words** and hence **can be overwritten by the user**. Hence, **you should always use `TRUE` and `FALSE`** .* » (§2.4)

La preuve est dans la liste des mots réservés (*R Language Definition* §10.3.3) : `TRUE` et `FALSE` y sont, `T` et `F` n'y sont pas. Après un `T <- 0`, toute condition écrite avec `T` devient fausse **sans le moindre message**.

`sqrt(-17)` donne `NaN` avec un avertissement. « *To work with complex numbers, supply an explicit complex part* » (§2.2) :

```
sqrt(-17+0i)  # le calcul est fait EN COMPLEXES
```

La règle générale : **R ne change pas de corps de nombres tout seul**. Entrée réelle, sortie réelle — quitte à valoir `NaN`. C'est l'utilisateur qui déclare l'intention.

« *Internally calculations are done as **double precision real numbers**, or double precision complex numbers **if the input data are complex**.* »

« *Perhaps unexpectedly, the number returned from the expression `1` is a numeric.* » (*R Language Definition* §3.1.1) C'est un **double**.

Le plus simple est le **suffixe** `L` : `1L`. Également `0x10L` (= 16), et `1e3L` = `1000L` — « *Note that the `L` is treated as qualifying the term `1e3` and not the `3`.* ».

Trois cas d'échec : `1e-3L` → **avertissement**, et c'est un **numérique** qui est créé ;
`1.L` → **avertissement** (point décimal inutile) ; `12iL` → **erreur de syntaxe**.

« *Except for the syntax, there is no difference between applying an operator and calling a function. In fact, `x + y` can equivalently be written ``+`(x, y)`. Notice that since `+` is a non-standard function name, it needs to be quoted.* » (*R Language Definition* §3.1.4)

Cela vaut aussi pour l'indexation : « *indexing is really done by functions, and one could have used ``[(x, 2)` instead of `x[2]`.* ».

Les conséquences pratiques : on peut **passer un opérateur en argument** (`sapply(1, `[, 1)`), **le redéfinir**, et **définir le sien** avec la syntaxe `%nom%`. C'est le socle du calcul sur le

langage (fiche 317).

La table des opérateurs (*R Language Definition* §3.1.4) est explicite : `&` est « *And, binary, **vectorized*** » ; `&&` est « *And, binary, **not vectorized*** ». Idem pour `|` et `||`.

`&` travaille **élément par élément** et rend **un vecteur de la longueur du plus long** (recyclage compris). `&&` rend **une seule valeur** : c'est la forme faite pour la condition d'un `if`, qui a besoin d'un scalaire. Voir fiche 308.

« ***matrices** or more generally **arrays** are multi-dimensional generalizations of vectors. **In fact, they are vectors that can be indexed by two or more indices** and will be printed in special ways.* » (§2.8)

Une matrice **n'est pas un type nouveau** : c'est un vecteur **muni d'un attribut** `dim` (fiche 303). D'où deux conséquences directes : `m[1]` — un seul indice — a un sens et rend le premier élément **du vecteur sous-jacent** ; et `length(m)` rend le **nombre total de cases**, pas le nombre de lignes.

Flashcards

Question	Réponse
La structure la plus simple de R ?	Le vecteur numérique — une collection ordonnée
Combien vaut la longueur de <code>3</code> ?	1 — un nombre isolé est un vecteur de longueur 1
Que fait <code>c()</code> ?	Concatène ses arguments bout à bout — donc aplatit
Que vaut <code>length(c(x, 0, x))</code> si <code>length(x) = 5</code> ?	11
Les quatre écritures d'assignation ?	<code><-</code> · <code>=</code> · <code>assign("x", ...)</code> · <code>-> x</code>
<code><-</code> est un raccourci de quoi ?	De <code>assign()</code>
Une expression seule, que devient sa valeur ?	Affichée puis perdue
L'arithmétique vectorielle est... ?	Élément par élément , avec boucle implicite
Longueur du résultat d'une opération binaire ?	Celle du plus long opérande
Que fait R du plus court ?	Il le recycle , éventuellement fractionnairement
Quand y a-t-il un avertissement ?	Quand la longue n'est pas un multiple de la courte
Le calcul se fait-il quand même ?	Oui — c'est ce qui rend l'erreur dangereuse
Le recyclage s'applique à quoi ?	À chaque opération binaire , pas à l'expression entière
Opération avec un vecteur de longueur 0 ?	Résultat de longueur 0 (depuis R 1.4.0)
Que vaut <code>range(x)</code> ?	<code>c(min(x), max(x))</code>
La formule de <code>var(x)</code> ?	<code>sum((x-mean(x))^2)/(length(x)-1)</code>
<code>var()</code> d'une matrice $n \times p$?	La matrice de covariance $p \times p$, lignes = observations
Différence <code>sort</code> / <code>order</code> ?	<code>sort</code> trie ; <code>order</code> rend la permutation qui trie
<code>max(a, b)</code> contre <code>pmax(a, b)</code> ?	Le plus grand du lot (longueur 1) contre position par position

Question	Réponse
Que vaut <code>sqrt(-17)</code> ?	<code>NaN</code> + avertissement
Comment forcer le complexe ?	<code>sqrt(-17+0i)</code>
En interne, R calcule en quoi ?	Double précision
<code>1</code> est-il un entier ?	Non , c'est un double
Comment écrire l'entier 1 ?	<code>1L</code>
Que vaut <code>1e3L</code> ?	<code>1000L</code> — le <code>L</code> qualifie <code>1e3</code>
Que donne <code>1e-3L</code> ?	Un avertissement , et une valeur numérique
Que donne <code>12iL</code> ?	Une erreur de syntaxe
Que vaut <code>1:30</code> ?	<code>c(1, 2, ..., 30)</code>
Priorité du <code>:</code> ?	Haute — <code>2*1:15</code> vaut <code>c(2, ..., 30)</code>
<code>1:n-1</code> pour <code>n <- 10</code> ?	<code>0 1 ... 9</code> — dix valeurs
<code>1:(n-1)</code> ?	<code>1 ... 9</code> — neuf valeurs
Séquence à rebours ?	<code>30:1</code>
Combien d'arguments a <code>seq()</code> ?	Cinq — jamais tous ensemble
Lesquels ?	<code>from</code> <code>to</code> <code>by</code> <code>length</code> <code>along</code>
<code>seq(2,10)</code> vaut ?	<code>2:10</code>
L'ordre des arguments nommés compte-t-il ?	Non
Longueur de <code>seq(-5, 5, by=.2)</code> ?	51
Pourquoi <code>seq(along = v)</code> ?	Sur un <code>v</code> vide il rend la séquence vide, <code>1:length(v)</code> rend <code>c(1, 0)</code>
<code>rep(x, times=3)</code> ?	Le motif répété : <code>1 2 3 1 2 3 1 2 3</code>
<code>rep(x, each=3)</code> ?	Chaque valeur répétée : <code>1 1 1 2 2 2 3 3 3</code>
Les trois valeurs d'un logique ?	<code>TRUE</code> , <code>FALSE</code> , <code>NA</code>

Question	Réponse
Comment naît un vecteur logique ?	D'une condition
Les six comparateurs ?	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>
Les trois opérateurs logiques ?	<code>&</code> <code>·</code> <code> </code> <code>!</code>
<code>&</code> contre <code>&&</code> ?	Vectorisé contre non vectorisé
Coercition d'un logique en arithmétique ?	<code>FALSE</code> → 0 , <code>TRUE</code> → 1
Compter les éléments vérifiant une condition ?	<code>sum(cond)</code>
En prendre la proportion ?	<code>mean(cond)</code>
Pourquoi ne jamais utiliser <code>T</code> / <code>F</code> ?	Ce sont des variables , pas des mots réservés
Que signifie <code>NA</code> ?	Not available — une donnée non disponible
La justification de la propagation ?	Spécification incomplète → résultat inconnaisable
Le bon test d'un <code>NA</code> ?	<code>is.na(x)</code>
Que vaut <code>x == NA</code> ?	<code>NA</code> partout — ce n'est pas un test
Qu'est-ce que <code>NaN</code> ?	Un résultat indéfini : <code>0/0</code> , <code>Inf - Inf</code>
<code>is.na(NaN)</code> ?	<code>TRUE</code>
<code>is.nan(NA)</code> ?	<code>FALSE</code>
Type par défaut d'un <code>NA</code> ?	Logique
Longueur de <code>x[NA]</code> ?	Celle de <code>x</code> — règles de l'index logique
Longueur de <code>x[c(1, NA)]</code> ?	2 — règles de l'index entier
<code>FALSE & NA</code> ?	<code>FALSE</code>
<code>TRUE NA</code> ?	<code>TRUE</code>
<code>NA == NA</code> ?	<code>NA</code>
<code>NA %in% c(1, NA)</code> ?	<code>TRUE</code> — <code>match</code> apparie les <code>NA</code>

Question	Réponse
<code>NaN</code> existe dans quel type ?	Le double seulement (et les composantes complexes)
<code>as.character(NaN)</code> ?	La chaîne <code>"NaN"</code>
<code>NA</code> caractère contre <code>"NA"</code> ?	Distincts → utiliser <code>NA_character_</code>
Existe-t-il un <code>NA</code> pour <code>raw</code> ?	Non
Comment <code>NA</code> s'affiche-t-il sans guillemets ?	<code><NA></code>
Guillemets acceptés pour une chaîne ?	Doubles ou simples ; affichage en doubles
Trois échappements courants ?	<code>\n</code> · <code>\t</code> · <code>\b</code>
Le backslash lui-même ?	<code>\\</code>
Échappement Unicode ?	<code>\unnnn</code> (4 chiffres) · <code>\Unnnnnnnn</code> (8)
Que fait <code>\0</code> dans une chaîne ?	Il termine la constante — interdit
Séparateur par défaut de <code>paste()</code> ?	Une espace
<code>sep</code> contre <code>collapse</code> ?	<code>sep</code> dans chaque élément ; <code>collapse</code> réduit à une seule chaîne
Que vaut <code>paste(c("X","Y"), 1:10, sep="")</code> ?	Dix étiquettes alternées : <code>"X1" "Y2" ... "Y10"</code>
Deux autres outils sur les chaînes ?	<code>sub</code> et <code>substring</code>
<code>%%</code> ?	Modulo
<code>/%</code> ?	Division entière
<code>%%*</code> ?	Produit matriciel
<code>%o%</code> ?	Produit extérieur
<code>%x%</code> ?	Kronecker
<code>%in%</code> ?	Appariement
<code>% %</code> ?	Coalescence nulle

Question	Réponse
Comment définir son propre opérateur ?	Syntaxe <code>%nom%</code>
Que vaut <code>`+`(x, y)</code> ?	<code>x + y</code> — opérateur = fonction
Et <code>`[(x, 2)</code> ?	<code>x[2]</code>
Que manque-t-il à la table d'opérateurs de R-lang ?	<code>!=</code> — donné au §2.4 de R-intro
Une matrice, c'est quoi au fond ?	Un vecteur indexable par deux indices ou plus
Une liste ?	Un vecteur général dont les éléments peuvent différer de type
Un data frame ?	Structure matricielle à colonnes de types différents, une ligne par unité d'observation
Une fonction, en R ?	Un objet , stockable dans l'espace de travail